

PadiHub MVP Backend Prompt Library v3.0

Prompt 1 — Backend Foundation, Database & Architecture

Objective

Build the complete backend foundation for the PadiHub MVP.

The frontend has already been designed and published using GoDaddy Airo App.

Do NOT redesign or recreate the frontend.

Instead, build the backend that supports the existing frontend.

Use modern, production-ready technologies.

Technology Stack

Build the backend using:

- Node.js
- TypeScript
- Express.js (or equivalent lightweight framework)
- MySQL
- Drizzle ORM
- REST APIs
- JWT or secure session-based authentication
- Environment variables for secrets

The backend should be clean, modular, scalable and easy to maintain.

Product Purpose

PadiHub enables trusted groups of people to save together through recurring contributions.

Each contribution cycle pays the pooled amount to one member according to the agreed rotation schedule.

Groups continue indefinitely until closed by the Group Leader or Platform Administrator.

There are:

- no funding targets
- no funding end dates
- no wallet functionality

- no customer funds held by PadiHub

Backend Responsibilities

The backend must provide:

- Database
- Authentication
- Business Logic
- REST APIs
- Scheduled Jobs
- Email Integration
- Payment Provider Integration
- Audit Logging

The backend must NOT generate frontend pages.

Database

Use MySQL.

Generate:

- Drizzle ORM schema
- Database migrations
- Seed scripts (where appropriate)

Use foreign keys and proper relationships.

Avoid duplicate data.

Database Tables

Users

Create a Users table.

Fields:

- id
- first_name
- last_name

- display_name
- email
- password_hash
- phone_number
- country
- currency
- trust_score
- subscription_status
- stripe_customer_id
- stripe_connected_account_id
- flutterwave_customer_id
- flutterwave_subaccount_id
- notification_preferences
- account_status
- email_verified
- active
- created_at
- updated_at

Do NOT include:

- Community Karma
- Passport
- Community DNA
- Marketplace
- Badges
- Achievements

Savings Groups

Fields:

- id
- name
- description
- leader_id
- country
- currency
- contribution_amount
- contribution_frequency
- maximum_members
- rotation_method
- current_rotation_position
- current_cycle
- status
- created_at

- updated_at

Do NOT include:

- funding_goal
- funding_progress
- end_date

Groups continue indefinitely.

Memberships

Fields:

- id
- user_id
- group_id
- role
- join_date
- status
- strike_count
- created_at
- updated_at

Contributions

Fields:

- id
- group_id
- member_id
- cycle_number
- amount_due
- amount_paid
- due_date
- paid_date
- payment_status
- provider_reference
- created_at
- updated_at

Rotations

Fields:

- id
- group_id
- cycle_number
- recipient_id
- scheduled_payout_date
- payout_status
- provider_transfer_reference
- completed_date
- created_at
- updated_at

Votes

Fields:

- id
- group_id
- proposal_type
- proposer_id
- proposal_text
- voting_deadline
- status
- created_at

Supported proposal types:

- payout_swap
- exceptional_request

No constitutions or governance framework.

Vote Responses

Fields:

- id
- vote_id
- member_id
- decision
- created_at

Notifications

Fields:

- id
- user_id
- type
- title
- message
- is_read
- created_at

Subscriptions

Fields:

- id
- user_id
- provider
- provider_subscription_id
- plan
- billing_status
- renewal_date
- created_at
- updated_at

Support Tickets

Fields:

- id
- user_id
- subject
- category
- description
- priority
- status
- created_at
- updated_at

Audit Logs

Fields:

- id
- user_id
- action
- entity
- entity_id
- ip_address
- created_at

Relationships

Implement proper foreign-key relationships.

User

↓

Membership

↓

Savings Group

↓

Contribution

↓

Rotation

↓

Vote

↓

Notification

REST API Structure

Prepare REST endpoints.

Authentication

/api/auth

Users

/api/users

Groups

/api/groups

Memberships

/api/memberships

Contributions

/api/contributions

Rotations

/api/rotations

Votes

/api/votes

Notifications

/api/notifications

Subscriptions

/api/subscriptions

Support

/api/support

Administration

/api/admin

API Standards

Return JSON.

Use:

GET

POST

PUT

DELETE

Follow REST best practices.

Return consistent HTTP status codes.

Validate all request bodies.

Implement central error handling.

Security

Use:

Environment variables

Password hashing (bcrypt)

JWT or secure sessions

Rate limiting

Input validation

SQL injection protection

CORS configuration

Helmet security middleware

Never expose:

API keys

Database credentials

Provider secrets

Project Structure

Organise the backend into:

/controllers

/routes

/services

/models

/database

/middleware

/utils

/config

/jobs

/emails

/integrations

Keep business logic inside services.

Keep controllers lightweight.

Deliverable

Generate a clean production-ready backend foundation that supports the published PadiHub MVP frontend.

No functionality outside the approved MVP should be included.

Do not generate frontend pages.

Focus only on the backend architecture, database, REST APIs and project structure.

PadiHub MVP Backend Prompt Library v3.0

Prompt 2 — Authentication, User Management & Security

Objective

Build the complete authentication and user management layer for the PadiHub MVP.

This prompt builds upon Prompt 1.

The frontend already exists.

Do NOT redesign or regenerate frontend pages.

Build only the backend services, APIs, authentication logic and security required to support the published frontend.

Authentication

Implement secure authentication.

Support:

- User Registration
- Login
- Logout
- Email Verification
- Forgot Password
- Password Reset
- Change Password
- Refresh Session

Passwords must never be stored in plain text.

Use bcrypt (or equivalent) hashing.

Session Management

Implement secure session handling.

Support either:

- JWT Authentication

or

- Secure HTTP-only session cookies

Use industry best practice.

Sessions should automatically expire.

Support logout from all devices.

Registration

During registration collect:

- First Name
- Last Name
- Display Name
- Email
- Password
- Mobile Number
- Country

Automatically assign:

- Trust Score
- Account Status = Pending Verification
- Subscription Status = Free Trial (or Free)

Users cannot use the platform until email verification is complete.

Email Verification

Generate secure verification tokens.

Tokens should:

- expire after configurable time
- only be usable once

After successful verification:

- activate account
- mark email_verified = true
- send Welcome Email

Login

Allow login using:

- Email
- Password

Return:

- Authentication Token
- User Profile
- Subscription Status

Reject login for:

- Unverified users
- Suspended users
- Deactivated users

Forgot Password

Generate secure reset tokens.

Tokens should:

- expire after configurable time
- be single-use

Send reset email.

Allow secure password change.

User Profile

Allow authenticated users to update:

- Display Name
- Phone Number
- Profile Picture
- Notification Preferences

Users may never modify:

- Trust Score
- Subscription Status
- Connected Payment Accounts

Supported Countries

Support:

United Kingdom

Currency:

GBP

Default Payment Provider:

Stripe Connect

Nigeria

Currency:

NGN

Default Payment Provider:

Flutterwave

Trust Score

Automatically initialise Trust Score.

Trust Score is updated ONLY by backend workflows.

Events affecting Trust Score:

Increase:

- Successful Contributions
- Completed Contribution Cycles

Decrease:

- Missed Contributions
- Suspensions
- Member Removal

Do NOT use:

- Karma
- Community activity
- Badges
- Marketplace
- AI scoring

User Roles

Implement Role-Based Access Control (RBAC).

Roles:

Member

Permissions:

- View own profile
- Join groups
- Leave groups
- Make contributions
- Vote
- View notifications

Group Leader

Additional permissions:

- Create Groups
- Edit Groups
- Invite Members
- Configure Group Rules
- Close Groups

Platform Administrator

Full permissions:

- Manage Users
- Manage Groups
- Manage Subscriptions
- View Audit Logs
- Manage Support Tickets

Authorization Middleware

Protect all private endpoints.

Only authenticated users may access protected APIs.

Implement middleware checking:

- Authentication
- User Status
- User Role

Reject unauthorized requests.

User APIs

Create REST APIs.

Authentication

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

POST /api/auth/forgot-password

POST /api/auth/reset-password

POST /api/auth/verify-email

GET /api/auth/me

Users

GET /api/users/profile

PUT /api/users/profile

DELETE /api/users/profile

GET /api/users/notifications

PUT /api/users/preferences

Validation

Validate:

Email format

Phone number

Password strength

Country

Required fields

Reject invalid requests.

Return meaningful validation errors.

Security

Implement:

bcrypt password hashing

Rate limiting

CORS

Helmet security headers

Input sanitisation

SQL injection protection

XSS protection

CSRF protection (if session authentication used)

Never expose:

Password hashes

Internal IDs

Secret Keys

Provider Credentials

Audit Logging

Automatically record:

User Registered

Email Verified

Login

Logout

Password Changed

Password Reset Requested

Password Reset Completed

Profile Updated

Account Suspended

Account Reactivated

Store:

- User

- Timestamp
- IP Address
- Action

Error Handling

Implement central error handling.

Return standard JSON responses.

Example:

Success

```
{  
  "success": true,  
  "data": {}  
}
```

Failure

```
{  
  "success": false,  
  "message": "Validation failed"  
}
```

Use appropriate HTTP status codes.

Deliverable

Generate a secure authentication and user management backend.

The implementation should support:

- secure authentication
- role-based permissions
- email verification
- password recovery
- profile management
- Trust Score foundation
- audit logging

The backend should be production-ready and integrate seamlessly with the existing GoDaddy Airo frontend.

Do not generate frontend pages.

PadiHub MVP Backend Prompt Library v3.0

Prompt 3 — Savings Groups, Memberships, Contributions, Rotation Engine & Governance

Objective

Build the core business logic for the PadiHub MVP.

This prompt implements the application's primary functionality:

- Savings Groups
- Membership Management
- Group Invitations
- Contribution Tracking
- Rotation Engine
- Governance Voting
- Trust Score Updates

The frontend already exists.

Do NOT regenerate frontend pages.

Generate only backend code, REST APIs, services, database logic and scheduled jobs.

Development Standards

Generate:

- TypeScript Interfaces
- Drizzle ORM Models
- Repository Layer
- Service Layer
- Controller Layer
- REST APIs
- OpenAPI / Swagger documentation
- Unit-test stubs

Keep controllers lightweight.

Business logic belongs inside Services.

Database access belongs inside Repositories.

Savings Groups

Implement CRUD operations for Savings Groups.

Each Group contains:

- Name
- Description
- Leader
- Country
- Currency
- Contribution Amount
- Contribution Frequency
- Maximum Members
- Rotation Method
- Current Rotation Position
- Current Cycle
- Status

Supported Statuses:

- Active
- Closed
- Suspended

Groups never have:

- Funding Goal
- Funding Progress
- End Date

Groups remain active until manually closed.

Group Creation Wizard

Support backend processing for the frontend wizard.

Step 1

Store:

- Group Name
- Description
- Country
- Currency

Step 2

Store:

- Contribution Amount
- Contribution Frequency

Supported Frequencies:

- Weekly
- Monthly

Store:

Maximum Members

Step 3

Rotation

Store:

- Rotation Method
- First Recipient
- Current Rotation Position = 1
- Current Cycle = 1

Supported Methods:

- Manual Order
- Random Order (generated once)

Step 4

Governance Rules

Store:

- Strike Threshold
- Suspension Threshold
- Voting Threshold
- Allow Payout Swaps

Step 5

Payment Provider

Automatically assign:

United Kingdom

Stripe Connect

Nigeria

Flutterwave

Step 6

Invitations

Generate:

- Invitation Token
- Invitation Link

Support invitations via:

- Email
- Shareable Link

Invitation tokens should expire after configurable period.

Membership Management

Support:

- Join Group
- Leave Group
- Accept Invitation
- Decline Invitation
- Remove Member

Membership Status:

- Pending
- Active
- Suspended
- Removed

Automatically update member counts.

Contribution Engine

Generate recurring contribution schedules.

Each Contribution contains:

- Member
- Group
- Cycle Number
- Due Date
- Amount Due
- Amount Paid
- Status

Contribution Status:

- Scheduled
- Due
- Paid
- Failed
- Missed

Contribution Services

Generate backend services for:

Create Contribution

Update Contribution

Mark Paid

Mark Failed

Mark Missed

Retrieve History

Rotation Engine

Implement automatic rotation management.

Maintain:

- Current Recipient
- Next Recipient
- Previous Recipient
- Rotation Position
- Cycle Number

When a cycle completes:

Advance:

Current Rotation Position

Increment:

Cycle Number

Create next Rotation record.

Groups continue indefinitely.

Rotation APIs

Generate endpoints for:

Retrieve Current Rotation

Retrieve Rotation History

Retrieve Next Recipient

Advance Rotation

Only backend workflows may advance rotations.

Governance

Implement simple governance only.

Supported Proposal Types:

- Payout Swap Request
- Exceptional Request

No constitutions.

No community governance.

No elections.

Voting

Create voting engine.

Workflow:

Create Proposal

↓

Notify Members

↓

Collect Votes

↓

Calculate Result

↓

Notify Members

Supported Results:

- Approved
- Rejected
- Expired

Trust Score

Trust Score changes ONLY through backend logic.

Increase:

- Successful Contribution
- Successful Cycle Completion

Decrease:

- Missed Contribution
- Suspension
- Member Removal

Never calculate Trust Score using:

- Community Karma
- Marketplace
- Passport
- AI
- Challenges
- Leaderboards

Notifications

Automatically create notifications for:

Group Created

Invitation Sent

Invitation Accepted

Contribution Due

Contribution Paid

Contribution Failed

Contribution Missed

Upcoming Payout

Payout Completed

Vote Required

Vote Closed

Member Suspended

Member Removed

Store:

- Recipient
- Notification Type
- Title
- Message
- Read Status
- Timestamp

Scheduled Jobs

Generate scheduled backend jobs.

Daily:

- Contribution reminders
- Overdue contribution checks
- Trust Score updates

Weekly:

- Remove expired invitations

Monthly:

- Generate new contribution schedules
- Advance completed rotations

REST APIs

Generate secure REST endpoints.

Groups

GET /api/groups

POST /api/groups

PUT /api/groups/:id

DELETE /api/groups/:id

Memberships

GET /api/memberships

POST /api/memberships

DELETE /api/memberships/:id

Contributions

GET /api/contributions

POST /api/contributions

PUT /api/contributions/:id

Rotations

GET /api/rotations

PUT /api/rotations/:id

Votes

GET /api/votes

POST /api/votes

PUT /api/votes/:id

Notifications

GET /api/notifications

PUT /api/notifications/:id/read

Audit Logging

Automatically record:

Group Created

Group Updated

Group Closed

Invitation Sent

Invitation Accepted

Member Joined

Member Removed

Contribution Created

Contribution Paid

Contribution Failed

Contribution Missed

Rotation Advanced

Vote Created

Vote Submitted

Vote Closed

Trust Score Updated

Deliverable

Generate a production-ready backend implementing:

- Savings Groups
- Membership Management
- Contribution Tracking
- Rotation Engine
- Governance Voting
- Trust Score
- Notifications

Generate clean, modular TypeScript code with Drizzle ORM, REST APIs, OpenAPI documentation and unit-test stubs.

Do not generate frontend pages.

Implement only the approved PadiHub MVP.

